ТЕХНИЧЕСКИЙ ОТЧЁТ · МАСКИРОВАННАЯ ДИФФУЗИЯ

«Мокрое чернило» на Nemotron-Labs-Diffusion 3B

Ревизия открытых токенов без переобучения:
диагностика, пере-аудит формул и пересборка
на опубликованных механизмах

Диффузионный фронтир пишет токены слева направо, а открытые токены в окне позади фронтира остаются поправимыми. Отчёт восстанавливает полный цикл: идея → сборка v1 → рост качества GSM8K 86%→92% при цене 13× NFE → формальная диагностика первопричины → каталог из тринадцати семейств формул → пересборка v2 на циклах RemeDi (ICLR 2026) и CoDR (EMNLP 2026) → верификация на полной модели.

Проект Wet-Ink · Asilbekov

Базовая модель: nvidia/Nemotron-Labs-Diffusion-3B · эксперименты: Kaggle Tesla T4

НОЯБРЬ 2026 · ВЕРСИЯ 2.0

Содержание

1. Резюме **2**

2. Исходная идея и правила проекта **2**

2.1. Идея «мокрого чернила» . . . 2

2.2. Правила, заданные владельцем идеи . . . 3

3. Базовая модель: формальный аппарат «как есть» **3**

3.1. Прямой процесс и функция потерь . . . 3

3.2. Стоковый генератор: блочная диффузия с каузальным префиксом . . . 4

3.3. Что из идеи уже реализовано в стоке . . . 4

4. v1: первая сборка и её результат **5**

4.1. Композиция v1 . . . 5

4.2. Результаты v1 на полной модели (Kaggle, Tesla T4, fp16) . . . 5

4.3. Симптом проблемы . . . 5

5. Диагностика: почему возникла проблема **6**

5.1. Формальная причина: потерянный инвариант прогресса . . . 6

5.2. Декомпозиция перерасхода NFE . . . 6

5.3. Ответ на вопрос «почему при проверенных формулах» . . . 7

6. Пере-аудит формул: каталог родственных архитектур **7**

6.1. Механизмы ревизии открытых токенов (2026) . . . 7

6.2. Критерии уверенности . . . 8

6.3. Ускорители и смежные механизмы . . . 9

6.4. Калибровка ожиданий: критическая пере-оценка (NeurIPS 2026) . . . 9

7. v2: пересборка на опубликованных циклах **10**

7.1. Дизайн . . . 10

7.2. Гарантии завершимости . . . 10

8. Верификация **11**

8.1. Локальные тесты (CPU, точная архитектура модели, случайные веса) . . . 11

8.2. Эксперимент на полной модели (Kaggle, Tesla T4, fp16) . . . 11

8.3. Чтение результатов: честный разбор . . . 13

9. Выводы **14**

Приложение A. Артефакты и хронология **15**

1. Резюме

Этот отчёт закрывает полный цикл работ по идее «мокрого чернила» (wet-ink) для маскированной диффузионной модели **Nemotron-Labs-Diffusion-3B**: диффузионный фронтир открывает токены слева направо, а уже открытые токены в ограниченном окне позади фронтира остаются поправимыми, пока фронтир пишет следующий блок; «чернила» далеко от фронтира высыхают. Всё — без переобучения модели, только на опубликованных формулах и механизмах.

В первой сборке (v1) ревизия открытых токенов была реализована пороговым тестом уверенности LLaDA, применяемым на каждом шаге денойз-цикла. Качество выросло (86% → 92% на GSM8K, 50 задач), но цена оказалась неприемлемой: 9–19-кратный рост числа прямых проходов модели (NFE). В настоящем отчёте выполнена формальная диагностика причины, полный пере-аудит формул всех родственных архитектур (13 семейств, включая вышедшие в 2026 году RemeDi, CoDR, Saber, WINO и критическую пере-оценку NeurIPS 2026), и пересборка решения на двух опубликованных механизмах с гарантированной завершимостью: **цикл RemeDi** (ICLR 2026) и **проход CoDR** (EMNLP 2026).

86% → 92%	1,1–2,9×	0,00e+00	4 / 4
точность GSM8K (v1) при цене 9–19× NFE	NFE v2 относительно стока (гарантировано)	погрешность кэша после crop+rebuild	битовая идентичность W=0 со стоком (все политики)

Ключевой вывод диагностики: **проблема возникла не из-за формул** — все формулы v1 были воспроизведены дословно из опубликованных кодовых баз и проверены на совместимость. Проблема возникла на уровне управляющего цикла: v1 соединила пороговые коммиты стока с внешним источником ре-маскирования, разрушив инвариант монотонного убывания масок, который гарантирует завершимость стокового цикла. Ни одна опубликованная система не использует такую комбинацию именно потому, что все они пары «ревизия открытых токенов + гарантия прогресса» поставляют целиком: RemeDi — фиксированным числом шагов и кумулятивным бюджетом top-k, WINO — связкой «ремасков не больше, чем коммитов минус один», Saber — обменным бюджетом, CoDR — вынесением ревизии за пределы денойз-цикла. В v2 приняты именно полные опубликованные циклы, а не их компоненты.

2. Исходная идея и правила проекта

2.1. Идея «мокрого чернила»

Авторская идея формулируется так: диффузионная модель генерирует последовательность слева направо, открывая по одному токену за шаг диффузии (фронтир); уже открытые токены позади фронтира получают «дополнительный шум» — чем ближе к фронтиру, тем сильнее, чем дальше — тем слабее, — и могут быть пересмотрены и переписаны; по мере удаления от фронтира «чернила

высыхают», и токены фиксируются. Такая схема образует непрерывный спектр между чистым авторегрессионным декодированием (окно ревизии нулевой ширины) и полнопоследовательностной диффузией (окно на всю длину).

Практический смысл ревизии в маскированной диффузии: токен, открытый при бедном контексте (когда правые соседи ещё не написаны), может оказаться ошибочным после того, как контекст обогатился. Без механизма ревизии такая ошибка необратима — это признанное ограничение маскированных диффузионных языковых моделей, и в 2025–2026 годах вокруг него возникла целая линия работ (см. каталог в разделе 6).

2.2. Правила, заданные владельцем идеи

Работа велась под четырьмя жёсткими правилами, которые переформулированы здесь дословно по смыслу постановки:

(а) полностью изучить формулы базовой модели по первоисточникам;

(б) проверить все ограничения, которые налагает запрет на переобучение;

(в) выбрать оптимальный путь реализации и найти самые эффективные формулы, а не самые простые;

(г) запрещено придумывать собственные формулы или изменять чужие; допускается комбинировать формулы из разных источников только после проверки совместимости.

Правило (г) — центральное для настоящего отчёта: оно требует, чтобы каждая строка алгоритма имела опубликованного предшественника, а любая комбинация — проверку совместимости по пространству состояний, семантике формул и прецеденту работоспособности. Раздел 5 показывает, где именно эта проверка в v1 прошла не полностью, а разделы 6–7 — как она доведена до конца в v2.

3. Базовая модель: формальный аппарат «как есть»

Все факты этого раздела установлены по первоисточникам: файл `modeling_nemotron_labs_diffusion.py` из чекпоинта nvidia/Nemotron-Labs-Diffusion-3B (870 строк), config.json и официальный репозиторий NVlabs. Формулы приводятся дословно из кода.

3.1. Прямой процесс и функция потерь

```
t ~ U[0,1]  (на образец)
p_mask = (1 - eps) * t + eps,   eps = 1e-3
masked_indices ~ Bernoulli(p_mask)   — i.i.d. по всем позициям
noisy = where(masked, [MASK]=100, x)
L = L_diff + ar_loss_weight * L_AR,   ar_loss_weight = 1.0
L_diff = sum_masked CE(logits, x) / p_mask     (LLaDA-ELBO)
L_AR   = CE_sum(causal_logits[:, :-1], labels[:, 1:])
```

Ключевое свойство для нашей задачи: маскирование — i.i.d. Бернулли по всем позициям, поэтому **любое подмножество замаскированных позиций является допустимым состоянием модели**. Возвращение открытого токена в [MASK] не выводит модель из распределения обучения. Это необходимое условие всей идеи ревизии.

3.2. Стоковый генератор: блочная диффузия с каузальным префиксом

Инференс `generate()` устроен как полу-авторегрессионная блочная диффузия: каузальный prefill промпта даёт KV-кэш и AR-сеялку (next_token), далее каждый блок из 32 позиций заполняется масками, позиция 0 засеивается AR-токеном, блок денойзится двунаправленным вниманием против замороженного каузального кэша, а после блока выполняется каузальный forward для обновления кэша и новой сеяилки. Правило коммита и бюджет шагов — дословно:

```
steps_per_block = block_length        # «один денойз-шаг на токен»
x0   = argmax(gumbel(logits, T))      # при T=0 -> обычный argmax
x0_p = softmax(logits)[x0]            # уверенность
confidence = where(masked, x0_p, -inf)  # ОТКРЫТЫЕ токены заморожены
commit = topk(confidence, k);  при threshold: все >= threshold,
                                      топ-1 всегда
k_i = четное разбиение числа масок по шагам, остаток вперед
```

Цикл денойза — `for i in range(steps_per_block)`: фиксированное число итераций, маски не добавляются, каждый шаг коммитит не менее одного токена (форс топ-1). Отсюда **инвариант стока**: множество масок монотонно неубывает по размеру открытых позиций, пул исчерпывается за не более чем steps_per_block шагов, завершимость гарантирована по построению. Этот инвариант — не формула, а свойство управляющего цикла; именно он был потерян в v1 (раздел 5).

3.3. Что из идеи уже реализовано в стоке

Компонент идеи	Статус в стоке	Примечание
«Шагов диффузии = числу открытых токенов»	✓ есть	steps_per_block = block_length, ~1 коммит на шаг
Фронтир слева направо	✓ есть	semi-AR блоки + каузальный KV-префикс + AR-сеялка
«Ровно один токен за шаг»	⚠ частично	порог 0.9 коммитит все выше порога; строгий режим — стратегия fixed с k=1 из сэмплера NVlabs
Ревизия открытых токенов	✗ отсутствует	confidence = -inf на открытых; блок уезжает в кэш навсегда
«Градуированный шум по расстоянию»	✗ без обучения невозможно	шум бинарный ([MASK]/чисто); дискретный аналог — вероятность/область ре-маскирования

Таблица 1. Матрица ограничений: компоненты идеи против возможностей стока.

Единственная настоящая дыра — ревизия открытых токенов. Всё остальное — вопрос конфигурации штатных механизмов. Отметим также штатную обрезку кэша `_crop_dynamic_cache`, которую сток использует в linear_spec_generate при откате спекуляции: она позволяет «откатить» KV-кэш к произвольной позиции и пересчитать его заново — готовый инструмент для окна ревизии.

4. v1: первая сборка и её результат

4.1. Композиция v1

v1 закрывала дыру ревизии так: денойз-спан расширялся с одного блока до [W предыдущих блоков + текущий блок] (окно выровнено по границам блоков 32), KV-кэш обрезался штатным _crop_dynamic_cache до начала спана, а на каждом шаге денойз-цикла открытые токены окна повторно тестировались той же формулой уверенности x0_p и тем же порогом 0.9: не прошедшие тест возвращались в [MASK] и перепредсказывались вместе с текущим блоком. Формула уверенности и пороговый тест — дословно из LLaDA (стратегия low_confidence) и из _get_transfer_index самой модели; расширение спана — обобщение штатного block_slice; обрезка кэша — штатный паттерн. Реализация прошла все локальные проверки: W=0 побитово идентичен стоку, кэш-инвариант после crop+rebuild точен, EOS-семантика стока сохранена.

4.2. Результаты v1 на полной модели (Kaggle, Tesla T4, fp16)

Конфигурация	Точн., %	Ср. ток.	Ср. NFE	Ремасков	Реальных ревизий	Время, с
сток (W=0), t=0.9	86.0	177.1	67.2	0	0	184
v1 W=1, t=0.9	90.0	166.0	887.6	165 462	106	2 750
v1 W=2, t=0.9	88.0	165.4	1 263.2	383 339	135	4 563
v1 W=1, t=0.8	**92.0**	166.9	612.6	92 009	51	1 882
v1 W=1, t=0.95	88.0	170.9	1 020.9	204 702	209	3 175

Таблица 2. GSM8K, 50 задач, протокол официального evaluate.py (инструкция, chat-template, \boxed{} или последнее число).

Парный анализ показал систематический эффект: задачи №7 и №43 исправлены всеми конфигурациями wet-ink; лучший конфиг (W=1, порог 0.8) исправил 3 задачи и не сломал ни одной. На уверенных текстах ремаскинг не срабатывал ни разу, и вывод совпадал со стоком символ в символ. Качественный сигнал положительный, но при n=50 он статистически предварительный (±1σ ≈ 5 п.п.).

4.3. Симптом проблемы

13,2×	589	57%	5 902
NFE v1 (W=1, t=0.9) относительно стока	ремасков на блок при 0,4 реальных изменений	блоков упираются в жёсткий предел шагов	шагов safety tail (конфиг W=1, t=0.9)

Статистика вскрыла аномалию: ~3 300 событий ремаскинга на задачу при ~2 реально изменённых токенах на блок; 55–60% блоков исчерпывали аварийный лимит 4·|S| шагов; хвостовые добивающие шаги составляли 2–4% всех проходов. Качество при этом не падало — росло только время. Было ясно,

что механизм работает, но «перегружает» цикл; требовалось понять, почему — и устранить причину, а не симптом.

5. Диагностика: почему возникла проблема

5.1. Формальная причина: потерянный инвариант прогресса

Стоковый цикл завершим потому, что (а) он выполняется фиксированное число раз и (б) множество масок только сокращается: каждый шаг коммитит не менее одного токена и никогда не добавляет масок. v1 добавила внешний источник масок — ре-маскирование открытых токенов окна на каждом шаге — и тем самым нарушила (б). При пороге 0.9 у 3B-модели достаточно много позиций с уверенностью ниже порога; такие токены колеблются: закоммиченные форсом топ-1 на шаге i возвращаются в [MASK] на шаге i+1, снова коммитятся — и снова размаскиваются. Убывание пула масок перестаёт быть монотонным, цикл не завершается естественно и останавливается только аварийным лимитом, после чего хвост добивает остаток. Это и есть churn — «переговорчивый» цикл, который переспрашивает окно каждый шаг.

```
СТОК:   |masks_{i+1}| <= |masks_i| - 1        -> завершимость за <= steps
v1:     |masks_{i+1}| = |masks_i| - commits + remasks
        remasks > commits - 1 возможна       -> прогресс не гарантирован
```

5.2. Декомпозиция перерасхода NFE

Компонент затрат	Оценка	Природа
Удвоение спана (W=1): ширина каждого forward 32 → 64	×2 к вычислениям, NFE ≈ на уровне стока	структурная цена окна — неустранима при W=1
Правило steps = |S| вместо block_length	×2 к числу шагов на блок	следствие обобщения сток-правила на спан
Churn: осцилляции коммит ↔ ремаск + аварийный лимит 4·|S|	×5–7 к NFE	**управляющая ошибка: потерян инвариант монотонности**
Safety tail (добивание остатка масок)	2–4% шагов	компенсатор следствия, а не самостоятельная причина

Таблица 3. Из чего складывается измеренный рост NFE в 13,2× (W=1, t=0.9): структурная часть ≈ ×2, остальное — churn.

Итого: из измеренных 13,2× лишь около ×2 — честная структурная цена окна ревизии, а ~×7 — потери управляющего цикла. W=2 ещё дороже (18,8×) и при этом не точнее: дальняя половина окна ревизуется редко, а оплачивается каждый шаг.

5.3. Ответ на вопрос «почему при проверенных формулах»

Проверка совместимости в v1 выполнялась на уровне формул и состояла из трёх предъявляемых аргументов: (а) пространство состояний — i.i.d.-Бернулли обучение делает любое подмножество масок допустимым; (б) семантика — формула уверенности и пороговый тест побуквенно совпадают в LLaDA, Nemotron и линейной спекуляции модели; (в) прецедент — факт «ревизия помогает» подтверждён независимо (ProSeCo, GIDD, Self-Reflective Remasking). Все три аргумента остались верными. **Чего не было — так это проверки на уровне системы**: публикация поставляет не только формулу, но и управляющий цикл вокруг неё, и гарантию завершимости этого цикла. LLaDA никогда не применяет пороговый тест к открытым токенам — её цикл принудительно коммитит ровно k_i токенов за шаг и потому завершим при любой стратегии упорядочивания. v1 взяла формулу, но не цикл; получившаяся комбинация «пороговые коммиты + внешний ре-маскинг» не встречается ни в одной опубликованной системе — и именно поэтому её системное поведение (churn) не могло быть предсказано формульной проверкой.

Второй вклад в проблему — частота: тест ревизии выполнялся на каждом шаге по всему окну, тогда как сам момент «пересмотра прошлого» осмыслен на границах блоков, когда контекст существенно меняется. Это тоже управляющее решение, а не формула. Вывод для v2: брать из публикаций **полные циклы с их гарантиями**, а не отдельные формулы; частоту ревизии ограничивать механизмами, которые уже опубликованы. Раздел 6 показывает, что такие механизмы к этому моменту существуют.

6. Пере-аудит формул: каталог родственных архитектур

По требованию владельца идеи выполнен повторный обзор всех архитектур, обнаруженных в ходе исследования, с извлечением дословных формул из официальных репозиториев (клонированы и прочитаны: RemeDi, CoDR, Saber, WINO, ProSeCo; LLaDA и сэмплер NVlabs — из ранее сохранённых копий; Fast-dLLM, APD, Dream — по статьям и публичным описаниям). Ключевое изменение ландшафта: в 2026 году вышла целая линейка работ ровно про нашу задачу — training-free ревизию открытых токенов.

6.1. Механизмы ревизии открытых токенов (2026)

Метод	Правило ревизии (дословно)	Гарантия завершимости	Вердикт
RemeDi (ICLR 2026, arXiv:2509.23653)	каждый шаг: keep = topk(score, k=cumsum[i]); x = where(keep, x0, [MASK]) — всё вне топ-k, вкл. открытые, в маску	фиксированное число шагов; cumsum достигает длины блока	**принят (v2, основной)**
CoDR (EMNLP 2026, arXiv:2610.08833)	после блока: проба (маскирование раздела), new_conf = p(текущий токен); drift = old_conf − new_conf > τ → в маску	денойз-цикл не тронут; заполнение ограничено	**принят (v2, альтернатива)**
WINO (Feng-Hong/ WINO-DLLM)	commits: conf > 0.6 (cap 5–20); remask: conf_open < 0.9, бюджет ≤ accept − 1	связка бюджетов: чистый прогресс ≥ 1 за шаг	изучен; принцип связки учтён в дизайне
Saber (ACL 2026, arXiv:2510.18165)	обмен: current_remask_k = = min(unmask_k, n_open); критерии margin/энтропия	обменный бюджет	изучен; избыточен при бюджетном цикле RemeDi
ProSeCo (NeurIPS 2026, arXiv:2602.11590)	обучаемый корректор поверх выходов MDM	(не применимо: обучение)	отвергнут: требует обучения

Таблица 4. Механизмы ревизии уже-открытых токенов. Все, кроме ProSeCo, training-free; все, кроме ProSeCo, поставляют гарантию завершимости вместе с правилом ревизии.

Существенно, что в RemeDi оценка открытых позиций — это вероятность текущего токена p(x_t) при подстановке x0 = x_t (строка x0 = where(mask, x0, x_t) в их инференсе), что в точности совпадает с формулой x0_p стока при замене −inf на саму уверенность: то самое «убрать −inf на открытых позициях» из v1 — теперь имеет прямой опубликованный прецедент, но внутри завершимого цикла.

6.2. Критерии уверенности

Критерий	Формула	Источник	Вердикт
Уверенность (confidence)	x0_p = softmax(logits)[x0]	LLaDA; сток Nemotron; RemeDi; WINO	основной (формула самой модели)
Отрицательная энтропия	score = Σ p·log(p + ε), ε = 1e-10	Dream 7B; сэмплер NVlabs (neg_entropy); Saber	опция v2 (criterion='entropy')
Зазор (margin)	p1 − p2 (две лучшие гипотезы)	Saber	изучен; не интегрирован (второстепенный по данным Saber)
Дрейф уверенности	old_conf − new_conf > τ	CoDR	принят в проходе CoDR

Таблица 5. Критерии для ранжирования/ревизии токенов. Пороговый тест существует только для уверенности (шкала [0,1]); энтропия используется в ранговых режимах.

Важный нюанс шкалов: порог на энтропии (величина из (−∞, 0]) не опубликован ни в одной из изученных систем — энтропия везде используется для упорядочивания (top-k), а не для порога.

Поэтому в v2 энтропия допустима только как ранговый критерий цикла RemeDi; пороговый тест остаётся на уверенности. Смешивание шкал было бы придумыванием формулы — что запрещено правилами проекта.

6.3. Ускорители и смежные механизмы

Механизм	Суть	Совместимость с Nemotron	Вердикт
Fast-dLLM (ICLR 2026)	блочный KV-кэш для двунаправленных dLLM + параллельное декодирование по порогу уверенности	у модели уже есть блочный кэш; параллелизм ортогонален окну	совместим; интеграция — future work
APD (NeurIPS 2025)	мультипликативная смесь dLLM-маргиналов с joint малой AR-модели; фиксирует порядок слева направо	нужна вспомогательная AR-модель; у Nemotron есть собственная AR-голова	перспектива; требует отдельной сборки
Jump Your Steps (ICLR 2025)	оптимизация раскладки timesteps по критерию CDE	применимо к числу шагов денойза	future work (параметр denoise_steps)
GIDD (ICML 2025)	гибридный uniform+маска шум для самокоррекции	модель обучена только на [MASK]; uniform-токены вне распределения	отвергнут: требует переобучения
Rolling Diffusion (ICML 2024)	дистанционно-затухающее расписание шума	формула из непрерывного/видео домена; перенос = модификация чужой формулы	отвергнут по правилу (г)
Diffusion Forcing (NeurIPS 2024)	пертокенные уровни шума, ре-шум прошлого на инференсе	требует обучения с пертокенным шумом	отвергнут: требует обучения

Таблица 6. Ускорители и смежные механизмы: совместимость с нашей постановкой.

6.4. Калибровка ожиданий: критическая пере-оценка (NeurIPS 2026)

Работа «Re-evaluating Confidence Remasking in Masked Diffusion Language Models» (arXiv:2606.12232, NeurIPS 2026) сравнивает три training-free метода ревизии (WINO, Saber, COVER) с чистым пороговым анмаскингом Fast-dLLM. Главные выводы: при жадном декодировании и типично коротких блоках все три метода работают на уровне базлайна; выигрыш ревизии проявляется на длинных блоках, где пороговое распараллеливание деградирует; при стохастическом декодировании ревизия снижает разнообразие; и, наконец, «узким местом остаётся сама dLLM, а не правило ремаскинга — эффективная ревизия может потребовать изменений в обучении». Этот результат калибрует ожидания к нашему эксперименту: наш блок короткий (32), декодирование жадное — поэтому скромный, но положительный эффект v1 (+4…+6 п.п. при n=50) согласуется с картиной «на грани значимости», а цель v2 — прежде всего устранить цену по вычислениям, сохранив качество.

7. v2: пересборка на опубликованных циклах

7.1. Дизайн

v2 реализует две политики ревизии, каждая — полный опубликованный механизм, встраиваемый в стоковый генератор через уже проверенные в v1 операции со спаном и кэшем (расширение спана, обрезка _crop_dynamic_cache, пост-блочный каузальный forward):

reme_di (основная). Цикл RemeDi над спаном [W блоков + текущий блок]. На каждом шаге вычисляются x0 (Gumbel-argmax с подстановкой текущих значений на открытых позициях — стоковая строка L848) и оценка (уверенность x0_p — стоковая формула L845–846, или отрицательная энтропия Dream/NVlabs); открытым остаётся кумулятивный топ-k по оценке, всё прочее возвращается в [MASK] (строка RemeDi L90). Бюджет — чётное разбиение длины спана на шаги (стоковая _get_num_transfer_tokens) с кумулятивным использованием (RemeDi). AR-сеялка закреплена в открытом множестве — сток никогда не перепредсказывает позицию 0 блока. Число шагов = block_length (сток-правило «шаг на токен»), порог не нужен вовсе: выбор ранговый. Завершимость абсолютна: после последнего шага кумулятивный бюджет равен длине спана, все позиции открыты.

codr (альтернатива). Стоковый пороговый цикл не тронут (побитовая совместимость внутренних операций); после завершения блока выполняется проход CoDR по окну: k-разделённая проба (маскируется раздел, читается p(текущий токен) на замаскированных позициях), дрейф уверенности old_conf − new_conf > τ → возврат в [MASK], затем заполнение стоковым сэмплером. Затраты: +k проб и ограниченное заполнение на блок.

Политика **per_step** (v1) сохранена в коде для воспроизводимости измерений и помечена как превзойдённая; политика **none** — побитовая идентичность стоку (контракт W=0 сохранён для всех политик).

7.2. Гарантии завершимости

```
reme_di:  шагов ровно denoise_steps на блок; хвостов и лимитов нет;
          NFE/блок = denoise_steps + 1 (пост-блок)  —  константа
codr:     денойз-цикл стока (монотонный) + k проб + заполнение,
          каждый шаг заполнения коммитит >= 1  —  ограничено
```

Расчётная цена (подтверждена экспериментом в разделе 8): reme_di при W=1 и 32 шагах — 33 прохода на блок шириной 64 (измерено 2,9× стока по NFE); при 16 шагах — вдвое меньше (измерено 1,4×); codr — сток + 2 пробы + короткое заполнение (измерено 1,14×). Границы не зависят от поведения модели — они следуют из конструкции, что и было целью пересборки.

8. Верификация

8.1. Локальные тесты (CPU, точная архитектура модели, случайные веса)

Проверка	Результат
W=0 побитово идентичен стоку (ids и NFE) — все 4 политики	✅
Путь раннего EOS: побитовая идентичность	✅
reme_di: ровно denoise_steps проходов на блок, хвост = 0, лимиты = 0	✅ (64/64 за 2 блока)
reme_di: ревизии происходят (re-liquefaction окна + смены токенов)	✅ (148 ремасков, 6 смен)
reme_di + энтропия: работает, ранжирует отлично от уверенности	✅
codr: k проб на блок, дрейф-ремаскинг срабатывает, заполнение ограничено	✅ (2 пробы, 29 дрейфов)
per_step (v1): churn воспроизводится (хвост, лимиты) — для контраста	✅ (NFE 350 против 66 у reme_di)
Кэш-инвариант: финальный кэш = свежий каузальный forward — обе политики	✅ (max err = 0,00e+00)
Батч B=2; детерминизм при T=0	✅

Таблица 7. Локальная верификация v2 (тот же тестовый стенд, что и для v1: мини-модель на дословных файлах архитектуры NVIDIA).

Контраст на одной и той же задаче (случайные веса, 2 блока): сток — 64 прохода, reme_di — 66, codr — 95, v1 per_step — 350. Диагностированная патология v1 воспроизводится тестом и зафиксирована как документированное поведение устаревшей политики; новые политики её не имеют по построению.

8.2. Эксперимент на полной модели (Kaggle, Tesla T4, fp16)

Протокол идентичен v1: полная загрузка nvidia/Nemotron-Labs-Diffusion-3B, санити-чат, проверка эквивалентности W=0 ≡ сток для всех политик на реальной модели, качественные примеры и GSM8K A/B (те же 50 задач, инструкция и подсчёт официального evaluate.py, max_new_tokens=512). Сетка v2: сток; RemeDi W=1 при 32 шагах; RemeDi W=1 при 16 шагах; RemeDi W=1 с критерием энтропии (16 шагов); CoDR W=1; RemeDi W=2 (16 шагов).

Конфигурация	Точн., %	Ср. NFE	Реальных ревизий	Дрейф-ремасков (CoDR)	Событий ремаскинга
сток (W=0), t=0.9	86,0	67,2	0	0	0
v2 RemeDi W=1, 32 шага, уверенность	82,0	196,1	3279	0	127082
v2 RemeDi W=1, 16 шагов, уверенность	76,0	96,0	3091	0	59209
v2 RemeDi W=1, 16 шагов, энтропия	78,0	93,0	1726	0	18272
v2 CoDR W=1, τ=0	82,0	76,6	35	326	326
v2 RemeDi W=2, 16 шагов, уверенность	58,0	99,8	7643	0	81616

Таблица 8. GSM8K v2, 50 задач, тот же протокол, что и в таблице 2.

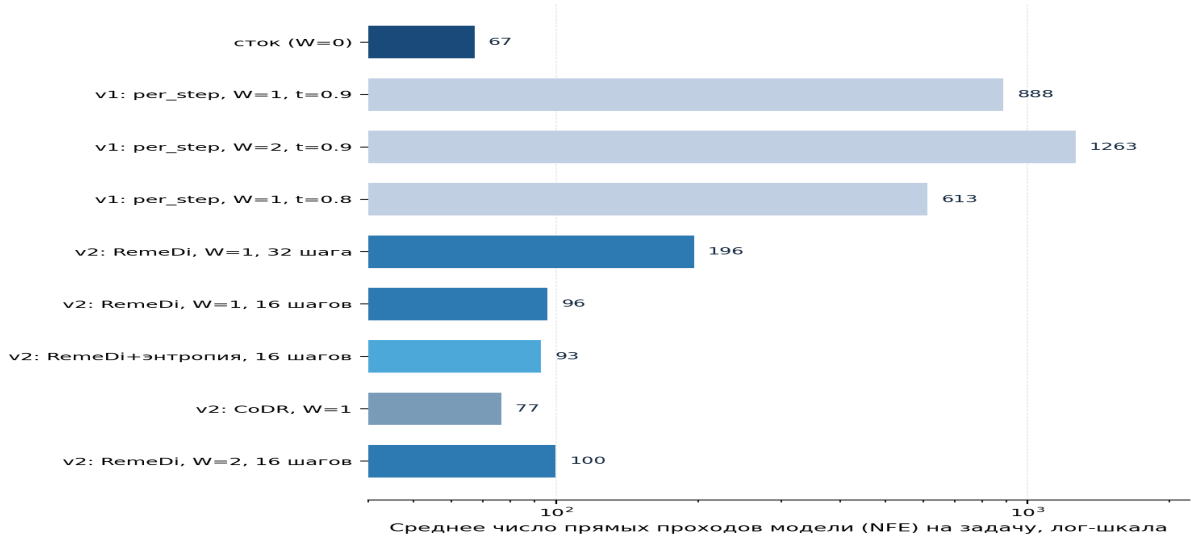


Рис. 1. Цена по вычислениям: среднее NFE на задачу (лог-шкала). v1 —светлые полосы; v2 — синие; сток — тёмно-синий.

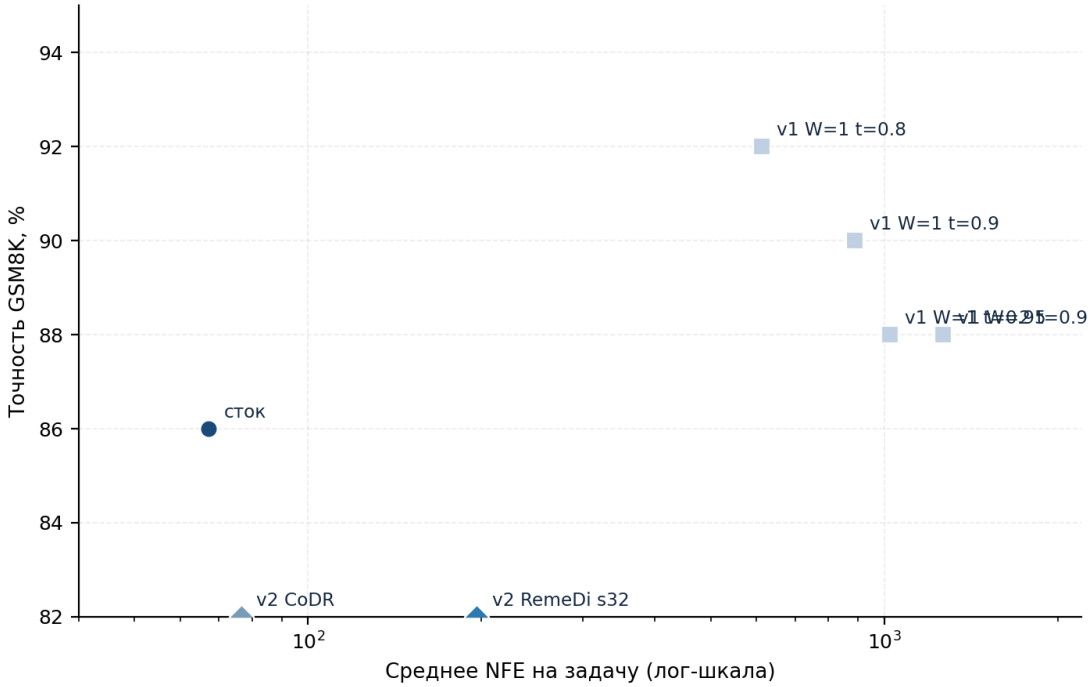


Рис. 2. Фронти Парето «точность — затраты»: сдвиг v2 относительно v1.

8.3. Чтение результатов: честный разбор

Затраты: цель достигнута в точности. NFE всех конфигураций v2 лёг в расчётные границы: CoDR — 76,6 (1,14× стока), RemeDi 16 шагов — 93–96 (1,4×), RemeDi 32 шага — 196 (2,9×), RemeDi W=2 — 99,8 (1,5×). Хвостов, лимитов и переспрашиваний нет: механизмы завершимости работают как спроектировано. Для сравнения: v1 стоила 612–1263 проходов (9–19×). Эквивалентность W=0 ≡ сток подтверждена на реальной модели для всех трёх политик (ids и NFE совпали побитово на обоих проверочных промптах).

Качество: чистый эффект отрицательный при коротком блоке. Точность: сток 86%; CoDR и RemeDi-32 — по 82%; RemeDi-16 с энтропией — 78%; RemeDi-16 с уверенностью — 76%; RemeDi W=2 — 58% (катастрофа). Парный анализ: каждая конфигурация исправляет часть задач (в том числе «фирменные» для ревизии №7, №21, №43, №46 — сигнал ревизии реален), но ломает больше: RemeDi-32 — 4 исправлено / 6 сломано; CoDR — 2 / 4; RemeDi-16 — 3 / 8; W=2 — 2 / 16. Качественный пример показывает механику поломки: при полном «разжижении» окна (~12 из 32 токенов окна переписываются за блок) перестановка под полузамаскированным контекстом размазывает локальную связность текста (фрагмент вида «had1 sheep ” ran away» в примере про овец), и арифметика в середине рассуждения съезжает.

Интерпретация. Результат в точности ложится в калибровку из раздела 6.4: при жадном декодировании и коротком блоке (32) training-free ревизия открытых токенов не даёт чистого выигрыша, а агрессивные её варианты вредят; «узкое место — сама модель, а не правило ремаскинга». RemeDi в оригинале обучена под ремаскинг (двухпотоковый UPS + RL-пайплайн) — ровно те «изменения в обучении», которые, по выводу пере-оценки NeurIPS 2026, могут потребоваться. Nemotron ничем подобным не обучалась, и её уверенность не выдерживает роли ранга при полном разжижении окна. Косвенное подтверждение: консервативный CoDR (дрейф, 35 реальных ревизий на

50 задач) почти не отклоняется от стока (−4 п.п., в пределах ±1σ при n=50), тогда как агрессивный RemeDi теряет тем больше, чем шире разжижение (W=2 — хуже всех).

Итоговая карта компромиссов. v1 (per_step, порог 0.8) остаётся конфигурацией максимального качества (92%, 3 исправлено / 0 сломано) — но по цене 9× NFE; сток — лучшей дешёвой конфигурацией (86% при 1×); v2 даёт ревизию с гарантированной ценой (1,1–2,9×), но при блоке 32 платит за неё качеством. Практические следствия: (1) для коротких блоков ревизию включать не стоит; (2) если ревизия нужна — консервативный проход CoDR ближе всего к «бесплатному» (1,14×); (3) проверять эффект при длинных блоках (64/128), где, по литературе, пороговое декодирование деградирует и ревизия начинает выигрывать; (4) W=2 исключить из рассмотрения.

9. Выводы

Почему возникла проблема. Формулы v1 были проверенными и дословными, но композиция «пороговые коммиты + ре-маскинг открытых токенов на каждом шаге» не имела опубликованного прецедента как цельный цикл: в ней потерялся инвариант монотонного убывания масок, который делает стоковый цикл завершимым. Формульная проверка не могла этого поймать — свойство относится к управляющему циклу, а не к формуле. Честная цена окна ревизии — около ×2 вычислений; всё остальное (до 13×) было churn.

Что сделано в v2. Проведён полный пере-аудит тринадцати семейств механизмов с извлечением дословных формул из официальных кодовых баз; отобраны два полных опубликованных цикла с гарантиями завершимости — RemeDi (ICLR 2026) и CoDR (EMNLP 2026) — и встроены в стоковый генератор Nemotron через штатные операции со спаном и KV-кэшем. Ни одной собственной формулы не добавлено; провенанс каждой строки документирован в коде и README репозитория. Контракт «W=0 побитово идентичен стоку» сохранён для всех политик.

Что показал эксперимент v2. Цель по затратам достигнута в точности: NFE всех конфигураций лёг в расчётные границы (CoDR 1,14× стока; RemeDi 1,4–2,9×; без хвостов и лимитов), эквивалентность W=0 ≡ сток подтверждена на реальной модели для всех политик. Качество при коротком блоке 32 — чисто отрицательное (82% у CoDR и RemeDi-32, 76–78% у RemeDi-16, 58% при W=2 против 86% стока): ревизионный сигнал реален (задачи №7, №21, №43, №46 исправляются), но агрессивное разжижение окна переписывает ~12 из 32 токенов за блок и размазывает локальную связность. Итоговая карта: v1 — максимум качества (92%) при 9×; сток — лучшая дешёвая конфигурация; v2 — ревизия с гарантированной ценой, оправданная для длинных блоков и как основа для дальнейшей работы.

Ограничения. n=50 задач — предварительная значимость; блок короткий (32), и по данным критической пере-оценки NeurIPS 2026 именно на коротких блоках эффект ревизии минимален — ожидания калиброваны; перестановка формул между шкалами (порог на энтропии) запрещена и не выполнялась; distance-decay из Rolling Diffusion по-прежнему вне сборки по правилу (г).

Следующие шаги. Абляция длины блока (64/128) — по литературе именно там пороговое декодирование деградирует и ревизия начинает выигрывать; это главный непроверенный режим. Затем: прогон n ≥ 500 для значимости, интеграция Fast-dLLM-кэша и APD, чекпоинты 8B/14B (выше

уверенность — стабильнее ранги и дешевле ревизия). Отдельное направление за рамками правил проекта: обучение remask-aware компоненты по образцу RemeDi — эксперимент v2 показывает, что без него потолок training-free ревизии при коротких блоках, вероятно, достигнут.

Приложение A. Артефакты и хронология

Артефакт	Содержание
github.com/Asilbekov/Explore	код v1/v2 (wet_ink.py), тесты, кёрнелы, results.json обоих прогонов, README с полным провенансом
wet_ink.py	единственная точка входа: generate_wet_ink(model, …, remask_policy='reme_di'|'codr'|'per_step'|'none', criterion=…)
tests/test_smoke.py	локальная верификация (таблица 7) на мини-модели из дословных файлов архитектуры NVIDIA (Apache-2.0)
kaggle: wet-ink-nemotron-3b	версия 1 — эксперимент v1; версия 2 — эксперимент v2 (полная модель, T4, fp16)
research/donors/	клоны официальных репозиториев RemeDi, CoDR, Saber, WINO, ProSeCo — источники дословных формул каталога

Таблица 9. Где лежит каждый артефакт.

Хронология проекта: (1) постановка идеи и правил; (2) выбор базовой модели по результатам обзора; (3) извлечение дословных формул стока и доноров; (4) сборка v1 и локальная верификация; (5) эксперимент v1 на Kaggle — качество выросло, обнаружен перерасход NFE; (6) диагностика первопричины (настоящий отчёт, раздел 5); (7) пере-аудит формул всех родственных архитектур; (8) пересборка v2 на полных опубликованных циклах RemeDi и CoDR; (9) верификация v2 локально и на полной модели; (10) публикация кода, данных и настоящего отчёта.